Machine Learning Approaches to Home Run Hitting Prediction

Group Members: Jonathan Sears, Jose Ramirez, Luis Madrigal

1. Introduction

The game of baseball is one of the most data-driven sports. Pitch counts, pitch velocity, batting averages, and velocity of the ball leaving the bat are a small portion of stats that teams keep track of in order to produce the best team from game to game. Analysis of these stats has become increasingly important in evaluating player performance in the game of baseball. As a result, predictive modeling has become a tool in understanding and forecasting player performance in baseball.

One of the biggest stats of baseball is the number of home runs a player or team hits in a season due to their ability to score points and change the momentum of the game. Predicting whether a hit will result in a home run can rely on factors such as launch speed, launch angle, pitch type, and pitch location. Even though these factors are well recorded, using them to help predict a home run is a much harder task. Reliable predictive models for this hard task help teams, analysts, and coaches produce the best team against the rest of the league.

This work aims to predict whether or not a hit will result in a home run depending on the pitched and batted ball data. The study uses the actual pitched and batted ball data from Kaggle. The data is from 2020-2021 and includes 10 plus features such as pitch name, pitch speed, and launch angle. To predict whether or not a hit resulted in a home run, three powerful machine learning algorithms (Logistic Regression, Random Forest, and SVM) were used.

2. Literature Review

In previous studies, Ramy Elitzur (2020) has shown that the use of data analytics in baseball has provided improvement in player selection and game management. This was done by asking the question whether the “organizational knowledge related to baseball data analytics” (Elitzur, 2020) has improved the Major League Baseball marketplace. A series of predictive modeling was used after identifying “Moneyball” (organizations that use data analytics) teams, their executives and the teams’ performance over time. The study shows that these teams saw a highly significant effect on win percentage (Elitzur, 2020).

Another study suggests that heuristics, a rule of thumb to make quick decisions, can be improved in baseball through the use of data analytics. This can be seen in the study done by Jae Hyeon Jo and Hyun Joung Jin (2026), where they analyzed pitches and the effects they had on batting statistics such as home runs, runs batted in, weighted runs created plus, and wins above replacement. Their results showed that traditional batting statistics are strongly associated with variations in pitch location, and advanced metrics such as batting runs, weighted runs created plus (measures a player’s offensive value), and win probability added resulted in a high association with zone out percentage (Jo & Jin, 2026).

3. Dataset/Preprocessing

For this analysis, the baseball data was gathered from Kaggle. The training data consists of pitch and batted ball data from Major League Baseball from 2020. This data set contains over 36,000 observations and more than 20 numerical or categorical features. Some examples of the features that will be used in the modeling consist of launch speed, launch angle, pitch type, pitch location, and park.

A MySQL server was set up to work with MySQL Workbench. This was done to be able to store the data in dataframes for easier manipulation for the models. The data was uploaded to a relational database in SQL, and this was connected to a Python notebook to be processed. This allowed for easy querying and processing of the data. The data was set up into Pandas dataframes as they provide ease to work with tools for the study, preparation, and modeling of the data.

Before the three models were conducted, the data was prepared by using multiple preprocessing techniques. In the considered dataset, there were no duplicates that needed to be removed from the data, but there were multiple instances of missing data. According to Figure 1, most of the instances of missing entries were from the features of launch speed and launch angle, but there were 6 instances of missing data for the feature of batted ball type. The missing data for the feature of batted ball type was replaced with an unknown since the column was categorical and we wanted to model the missingness as a separate category. For the missing entries of launch angle and launch, the row where the missing entry was dropped, because after dropping, there were over 25,000 observations left. Also, with these entries we assumed that the data was not recorded because it resulted in a walk or a strikeout, and we wanted the dataset to consist of hits that were put in play.

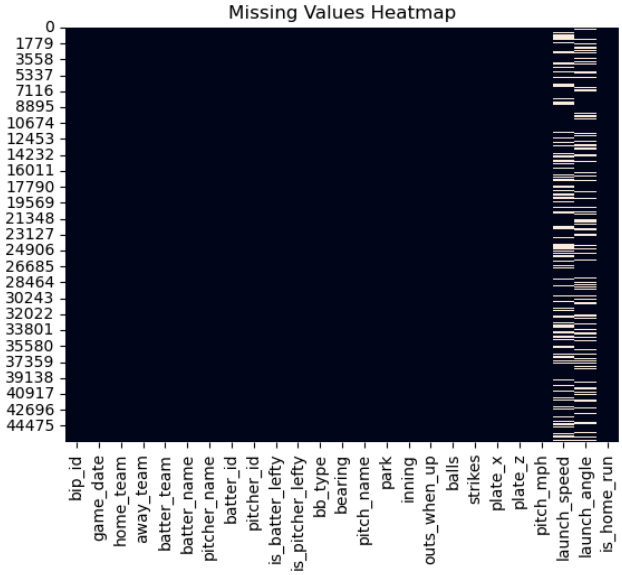

Figure 1: Heat map of missing data

After the data was cleaned of missing data, the data set was evaluated for insignificant features to the model. Columns that had high cardinality and low variance were dropped, which included the feature of a unique identifier of ball in play (bip_id), batter id, pitcher id, batter name, pitcher name, and game date. These columns were removed to avoid poor generalization. After removing features with high cardinality or low variance, the model still contained categorical features such as team, pitch name, or batted ball classification that needed to be addressed. All categorical features were then addressed by introducing dummy variables.

4. Data Exploration

After completing the data preprocessing, many summary statistics were calculated to obtain a general idea of the data set. Using pandas functions such as head(), info(), and describe(), we obtained a basic knowledge of how the dataset was organized and the types of variables present. The dataset contains a mix of numerical, categorical, and binary variables, with our chosen output variable, is_home_run, being binary as there are only two possible states. When calculating the mean for our target variable, we find that the mean is 0.0529. This indicates that only around 5% of observations are home runs, which shows that this is a very rare occurrence in baseball.

Plots were also used during the exploration to better visualize the dataset and identify patterns in the data.

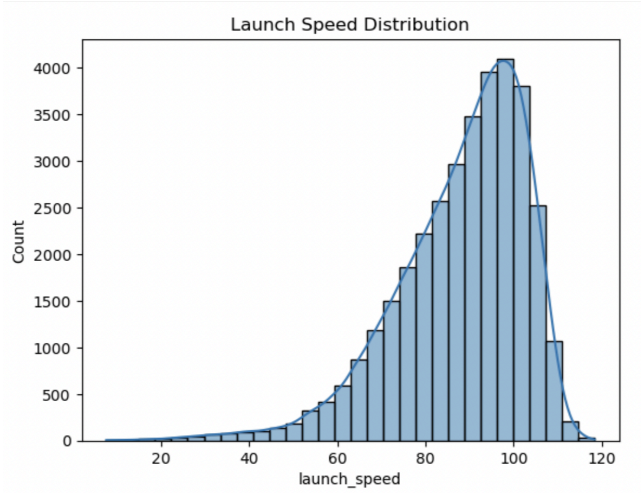

Figure 2: Histogram of launch speed

When looking at variables such as launch speed, it is evident that the majority of balls being hit exit with a speed between 60 and 120 mph, with a mean speed of 88.0 mph. The graph shows a tail to the left for a significant number of balls that were hit weakly. These data points could be considered outliers, but were included since weak hits are a common occurrence in baseball.

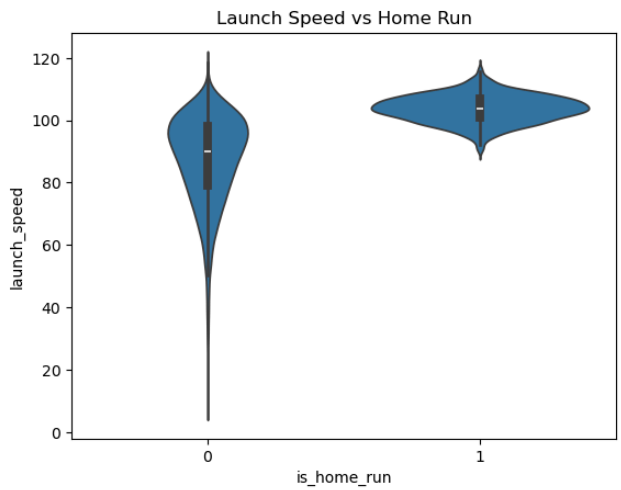

Figure 3: violin plot launch speed and homerun

The violin plot of launch speed vs. home run shows that for a home run to be hit, the ball must be hit with speed, with the majority of home run balls being hit having a launch speed greater than 100 mph. However, this plot also shows us that the launch speed is not the only important factor of a home run, since many balls hit with speeds over 100 mph were not home runs.

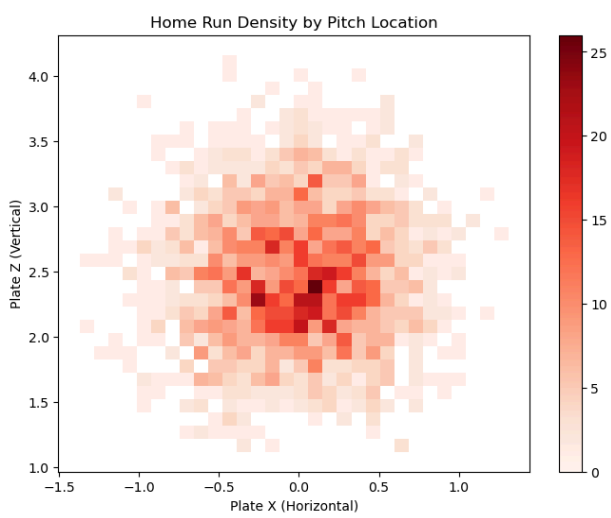

Figure 3: heatmap of pitch location and home runs hit

On the other hand, the incoming pitch also can have an effect on the result of the at bat, as pitches down the middle of the strike zone are easier to hit with good contact, and this can be visualized with the plot above. This shows the importance of the pitching location when trying to determine if a batted ball will be a home run.

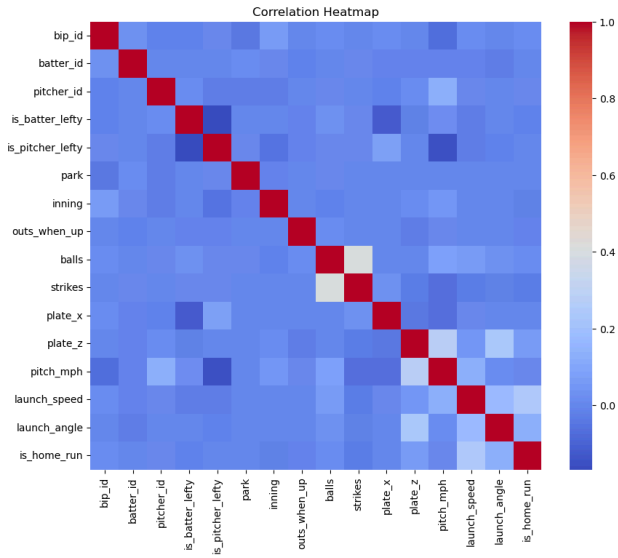

Figure 4: correlation heatmap between variables

When analyzing the possible correlation among the numeric variables in the baseball data, we observe a very weak linear correlation across the board, represented by the blue color in the heatmap. This suggests that nonlinear relations exist among these predictors. This tells us that nonlinear models will need to be used to be able to develop an effective regression model for this data.

5. Modeling/Evaluation/Validation

a) Support Vector Machine (SVM)

An SVM classifier was selected to determine whether a hit ball would result in a home run. This was chosen because it can model nonlinear relationships, which, as seen in the data exploration, was necessary given the very low linear correlation in the selected dataset. The data was split into a 70% train set and a 30% testing set to ensure the model is evaluated on data not used in creating it. To create the model, the SVM classifier was trained using a Radial Basis Function (RBF) kernel, which allows the SVM classifier to create curved decision boundaries, enabling it to detect nonlinear correlations among the variables. This complex kernel allows the SVM model to fit the complex data dealt with. Due to the class imbalance present in our data, a ‘balanced’ class weight was selected, which adjusted the weights taking into account the class frequency.

The resulting model was able to achieve an Area under the Curve (AUC) score of 0.92, which gives it an accuracy of around 90%. This figure, however, can be misleading, as given the rarity of home runs, it is easy for the model to achieve a high accuracy. For this reason, the confusion matrix was also calculated to better analyze the model's performance.

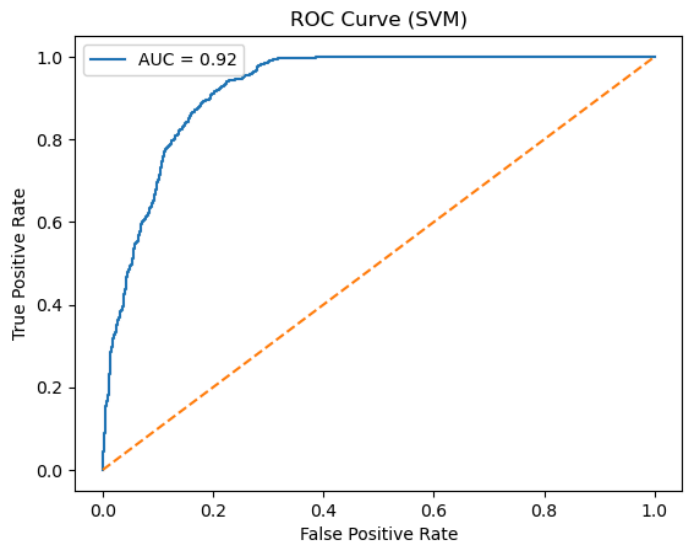

Figure 5: ROC Curve SVM Model

When looking at the confusion matrix in Table 1, it is clear that the model struggles with accurately predicting home runs, with a recall of 62% predicting actual home runs. This means that the model tends to overpredict home runs, which occurs given the rarity of these events, as many actual home runs were predicted not to be so. Additionally, some home runs are missed, further showing the challenge of accurately predicting when these events will occur.

	Predicted Not Home Runs (0)	Predicted Home Runs (1)
Actual Not Home Runs (0)	6750 (TN)	613 (FP)
Actual Not Home Runs (1)	150 (FN)	247 (TP)

Table 1: Confusion Matrix for SVM

Five-folds of cross-validation (CV) were performed to assess model stability, resulting in an average CV score of 0.9016, a score that is similar to the model's accuracy. This shows the model is stable and not overfitting. Overall, the model presented a high accuracy even when cross-validation was performed, but this figure was inflated given the nature of the data.

b) Logistic Regression

Not only was an SVM classifier selected to model the baseball data, a logistic regression was selected to predict whether a hit was classified as a homerun using pitched and batted data. A balanced logistic regression model was chosen to model the data as the target variable is binary and unbalanced, returns a predicted probability of class membership, and it handles continuous variables well such as launch speed and launch angle. Before initializing the logistic regression model, the data was split into training and test sets. The model was then fitted on the scaled input features as this made comparing the coefficients more realistic. The top and lowest

5 coefficients of the logistic regression are shown below in Table 2. The coefficients identified as the most influential predictors were launch speed, launch angle and if a batted ball is a fly ball while features such as hits bearing to the left, pop ups, ground balls, if the home team was Oakland lowered the likelihood of it being a home run.

Feature	Coefficient
Launch Speed	5.61
Launch Angle	2.77
Batted Ball Type (Fly Ball)	1.73
Batted Ball Type (Line Drive)	1.69
Home Team (NYY)	0.24
Away Team (ATL)	-0.21
Bearing Left	-0.28
Home Team (OAK)	-0.33
Batted Ball Type (Pop Up)	-1.71
Batted Ball Type (Ground Ball)	-2.07

Table 2: Coefficient Table

The logistic regression model was able to achieve an AUC score of 95% and overall accuracy score of 88% representing that the model is able to distinguish home runs from non-home runs. The model also had a high score of recall at 91% representing that most home runs were properly identified, but it had a low precision score of 29% which suggests that there were a lot of false positives. These scores represent that the model is able to rank home run probability but is less effective in classifying home runs.

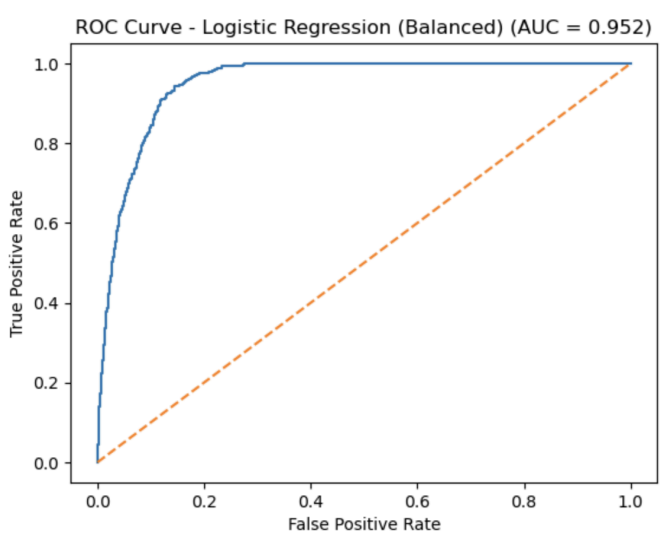

Figure 6: ROC Curve of Logistic Regression

A 5 fold cross validation was computed to determine the stability of the model. The average CV score was 95% with the minimum score of 0.951 and a maximum score of 0.958. This represents that there is no evidence of overfitting, strong generalization and that the model is able to distinguish home runs from non home runs across different folds.

Overall, the logistic regression performed well by being able to distinguish between home runs and non home runs and reveal key features such as launch speed and launch angle, but it had limitations due to its inability to capture nonlinear relationships within the data. To address these limitations, a more flexible model will be considered such as random forest. Random Forest models can capture nonlinear relationships between the features and handle continuous and categorical variables.

c) Random Forest

The third and final classifier chosen for modeling the dataset was the Random Forest Classifier. In contrast to the prior models, Random Forest did not require the input features to be scaled and required no preemptive consideration of the class imbalance due to its utilization of decision trees. To establish a baseline, the model was first initialized with default hyperparameter values before fitting on the training data. Doing this resulted in performance measures (accuracy, precision, recall, etc.) all equaling 1.0 when used to predict for the training dataset. This meant it significantly overfit on the training data, further backed by the resulting predictions on the test data. Despite getting a 0.97 accuracy, recall and F1-Score came in at under 0.6.

To tune the model in hopes of improving performance and resolving overfitting, GridSearchCV was used with an established grid of ranges of values for the following hyperparameters: “n_estimators”, “criterion”, “max_depth”, “min_samples_split”, and “max_features”. While time consuming (216 possible combinations considered based on the given values), it was determined that setting “max_depth” to 10, “n_estimators” to 5 and keeping the remaining hyperparameters at their default values generated more varied measurements with the training predictions, meaning the model was no longer plainly overfit. However, it had the same issue of performing poorly on precision and F1-Score on the test data.

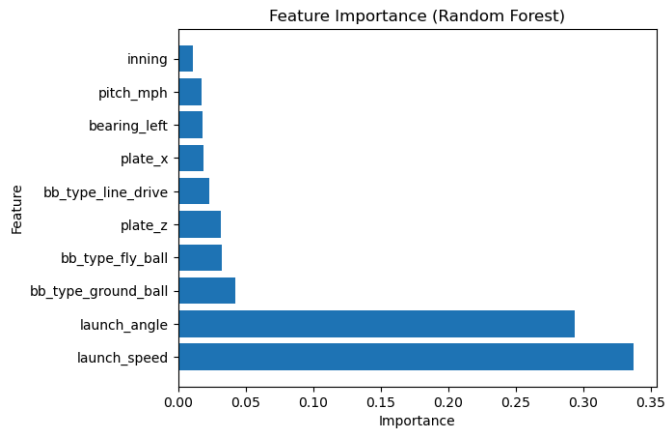

Figure 7: Feature Importance of Random Forest

Using built-in model attributes with the modified hyperparameter values, the top 10 important features were identified to try to improve performance as well as the cost when it came to runtime and hardware usage with the large amount of encoded features. With “launch_speed” already identified as a strong indicator in pre-processing and ranking first with feature importances, it was selected in combination with “launch_angle”. The model was then refit on these two features.

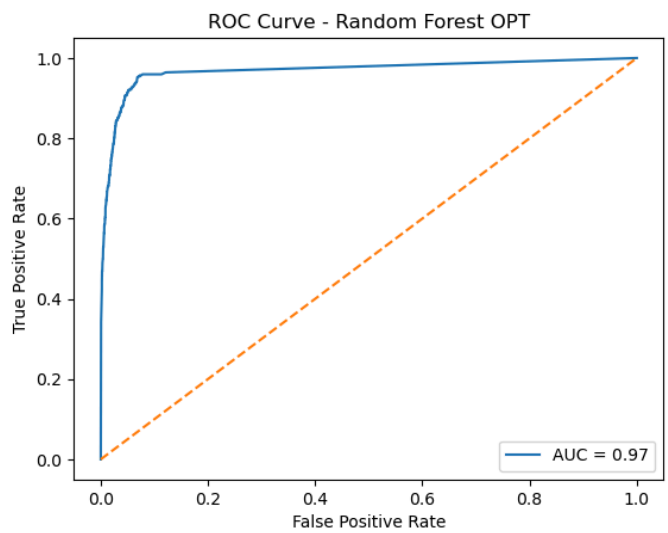

Figure 8: ROC Curve of Random Forest

Running this model iteration on the test data generated an accuracy of 0.97. While there was a drop in precision (down from 0.87 with the base model to now 0.76), it showed significant improvement on recall and F1-score (coming in at 0.67 and 0.71 respectively). This was a much more reasonable balance among the core performance metrics. To reinforce that the model was finally optimized, the AUC score was obtained with a value of 0.97 and the above ROC curve was generated, showing that the model was not as prone to random guessing.

6. Results

Figure 9: Model Performance Metrics

Considering all three models, as shown in Figure 9, Random Forest had the highest accuracy, precision, AUC, and F1-Score categories mentioned. Its high precision and F1-Score indicate a low false-positive rate and good performance balance, respectively. The logistic regression model had the best recall, meaning it did better at minimizing false negatives. The SVM model required the most hyperparameters set, with four. The important features for Random Forest closely resembled the features with highest coefficients for SVM and Logistic Regression, with “launch_angle” and “launch_speed” being the top two.

With our original goal in mind, all the models had relatively high accuracy and AUC. This indicates they were each good for classifying whether a hit in baseball is a homerun. That being said, each model still has areas in which they can improve before being used to definitively predict with certainty.

Some issues encountered included data outliers, such as the low values mentioned prior for the “launch_speed” feature. As this was one of the most important features among all the models in assisting predict the target, there was concern on it negatively impacting the results. This did not appear to affect accuracy or AUC negatively as all models performed well with this metric. However, this is likely a factor for the variability in the other performance metrics. As a group, it was discovered how important/difficult it is to obtain “perfect” data. Even without our data missing a lot of values, we had to adapt it differently based on each model chosen and the process for developing the models was non-linear in that data had to be explored partially before beginning preprocessing and explored even further once input into the model.

REFERENCES:

Elitzur, R. (2020). *Data analytics effects in Major League Baseball. Omega, 90*, 102001. https://doi.org/10.1016/j.omega.2018.11.010

Jo, J. H., & Jin, H. J. (2026). *Strategic pitching decisions and the role of heuristics in Major League Baseball. Psychology of Sport and Exercise, 84*, 103081. https://doi.org/10.1016/j.psychsport.2026.103081

Pedregosa et al. (2011). *Scikit-learn: Machine Learning in Python.* JMLR 12, pp. 2825-2830. https://scikit-learn.org/stable/index.html

Contributions and Key Lessons Learned

Jonathan Sears- In the project, I contributed to the coding of cleaning the data. This helped me learn the steps needed in order to have the best data set to model on. I learned to start with the repeating observations, then the missing observations, and finally perform feature selection by removing features with high cardinality or low variance. I also learned how to determine the best way to deal with missing observations through doing the preprocessing. I learned that it is best to drop the observation in most cases, but sometimes it is best to impute data depending on the reason why they are missing and how much missing data there is. I also contributed to the modeling and evaluation of the logistic regression model, which allowed me to learn the benefits of logistic regression. It helped me learn how logistic regression produces predicted probabilities and does not predict classes right away. It chooses the class based on the predicted probability that is produced and the decision rule that is set. The decision rule should be set based off the goal of the project. For example, it should be set higher if you want fewer false positives and lower if you want fewer false negatives. I finally learned that ridge and lasso regression should only be used if the features are numerical.

Luis Madrigal - For the project, I contributed to setting up the connection between MySQL and Jupyter Notebook. It was the first time I had to work with SQL and Python together; this was a great experience for me to learn the link between MySQL and other services. This enabled me to be able to import the data into SQL and continue our analysis in Jupyter. I was later able to contribute to the data exploration, generating plots that allowed us to see the patterns present in the data. This was a challenge as we had to understand what the patterns meant and how to deal with the nonlinear relationships present. After the exploration, I was able to contribute to creating an SVM model for our data. I had to learn to understand the difference in SVM models and how to pick the one that would better fit the shape of the data in our set. This allowed me to connect what we learned in lectures to the possible applications it might have, as when putting the concepts into practice, you learn them in a different way.

Jose Ramirez - My contributions for the project included setting up/assisting with group members navigating the GitHub for collecting/combining individual coding contributions as well as establishing the Random Forest model to predict home runs based off of the data. I also generated relevant visuals for the vital steps in this model's creation and for its performance. Lastly, I was responsible for collating the performance for each model and doing comparative analysis. A key learning opportunity for me with this project included understanding how to utilize MySQL to create a DATABASE for loading in datasets via files into a TABLE that could then be manipulated/utilized in Jupyter Notebook. Additionally, I learned more about how cyclical the model development process can be in terms of revisiting/revising input features and hyperparameters for models in order to generate better performance. Furthermore, I realized the importance of considering intrinsic aspects like interpretability and scalability during development to improve model longevity. This was achieved by balancing the reduction of the amount of input features needed and selecting less technical/complex hyperparameters to achieve similar or better performance.